

Chapter 6

Classes and Objects: Model DNA Samples

Learn object-oriented basics by turning a biological sample into a Python object.

EDOS learning idea

We will use biology and DNA examples because they are concrete, visual, and easy to experiment with. Type the examples yourself, change one thing, run again, and observe what changed.

What you will be able to do

- Understand classes and objects without jargon overload.
- Create a simple class with `__init__`.
- Store object attributes.
- Create methods that use object data.
- Model a DNA sample with reusable behavior.

Classroom rhythm

Read a small idea -> run the example -> change one value -> predict the result -> run again -> explain what happened in your own words.

6.1 What problem does a class solve?

Imagine every sample has an ID, a species, a condition, and a DNA sequence. You could keep four separate lists, but they can become messy. A class lets you describe what one sample should contain and what it should be able to do.

A class is a blueprint. An object is one thing made from that blueprint. A lab form is the blueprint; each completed form is an object.

6.2 Your first class

```
class DNASample:
    def __init__(self, sample_id, sequence):
        self.sample_id = sample_id
        self.sequence = sequence

s1 = DNASample('S01', 'ATGCGT')
print(s1.sample_id)
print(s1.sequence)
```

`__init__` runs when you create an object. `self` means the current object. `self.sample_id` stores information inside that object.

6.3 Add methods

```
class DNASample:
    def __init__(self, sample_id, sequence):
        self.sample_id = sample_id
        self.sequence = sequence.upper()

    def length(self):
        return len(self.sequence)

    def gc_percent(self):
        gc = self.sequence.count('G') + self.sequence.count('C')
        return gc / len(self.sequence) * 100

s1 = DNASample('S01', 'ATGCGCGTAA')
print(s1.length())
print(s1.gc_percent())
```

A method is simply a function inside a class. It can use data stored on `self`.

6.4 Multiple objects from one blueprint

```

samples = [
    DNASample('S01', 'ATGCGT'),
    DNASample('S02', 'GGGCCC'),
    DNASample('S03', 'ATATAT')
]

for sample in samples:
    print(sample.sample_id, sample.gc_percent())

```

Each object keeps its own data while sharing the same methods.

6.5 Add metadata and a summary method

```

class DNASample:
    def __init__(self, sample_id, species, sequence):
        self.sample_id = sample_id
        self.species = species
        self.sequence = sequence.upper()

    def summary(self):
        return f'{self.sample_id} | {self.species} | {len(self.sequence)} bases'

s = DNASample('S01', 'E. coli', 'ATGCGT')
print(s.summary())

```

Classes are useful when data and behavior naturally belong together. But do not force everything into a class. Simple scripts and functions are often enough.

6.6 When should a beginner use classes?

Use a class when you repeatedly manage many objects with the same fields and behaviors. For one tiny calculation, a function is usually simpler. Good Python is not about using the fanciest feature; it is about making the program clear.

Try it yourself

These are practice prompts, not a test. If your code breaks, that is useful information. Read the error, change one thing, and try again.

1. Create a Gene class with name and sequence attributes.
2. Add a length() method to Gene.
3. Add an a_count() method.
4. Create three Gene objects and loop through them.
5. Explain the difference between a class and an object using a biology analogy.

Tiny debugging habit

When something fails, check spelling, quotation marks, brackets, colons, and indentation first. Then print intermediate values so you can see what Python sees.

Chapter summary

- A class is a blueprint.
- An object is one instance of that blueprint.
- `__init__` stores starting data.
- Attributes hold object data.
- Methods are functions attached to objects.
- Classes are useful when data and behavior belong together.

Next step

Open Notebook o6 in EDOS Jupyter and run the matching exercises. Then continue to Chapter 7.

EDOS - Educational Desktop Operating Simulator
Copyright by Dr. Yash Munnalal Gupta